A Runtime Control Layer Under Hermes and Goose: What Scripts Showed and Live Models Exposed

Patrick Dugan
Morality Lab

October 2026

Abstract

Agent runtimes differ in how they call tools, retry, delegate and compact context, and the controls that keep an agent within what its user approved are usually written into one runtime at a time. We build one control plane that sits under two runtimes, Hermes Lite (Python) and goose (Rust): a hash-chained ledger that is the only source of truth, policy engines combined by conjunction, a tool gateway that is the only path to effects, a model proxy, an egress sink, and a controller thread above the agent loop that can only reduce authority. On BitAgent's development scenarios, with scripted policies, every run through each of four runtime arms is byte-identical to the same run without a runtime, the adversary lands no unauthorized effect, every gate knockout is caught, and the two modified runtimes write the same decision projection of the ledger for the same calls. The first live models found what scripted ones could not: a frozen prompt that made the model answer in text, protocol noise on a shared stdout, a turn cap one runtime ignores, and a soundness gap, an execute move sent through the tool registered as having no effect, which the host executed off the consequential path. Every hard gate stayed at zero once each was fixed. Qualifying a small model as the controller showed more. The first rule's positives could be caught by a lookup of the visible admissible moves; a counterfactual replay misstates a controller that reads its own earlier turns, which live intervened where the replay had passed or held; and our first live scorer labelled every refused turn from a missing proposal, so every refusal scored as right. After each was corrected, a 1-bit 8B controller with no brief fails the stricter rule (it mostly intervenes and almost never holds). With a short domain brief the same model passes it on the development seeds and on two held-out sets, but by holding about three quarters of the turns: precision 0.194 against a base rate of 0.165, F1 0.320 against `always_hold`'s 0.283, and fewer completed runs, while the rule its brief states, applied to the same turns, scores F1 0.801. A deterministic engine implementing that rule refuses with precision 1.000 and recall 0.896 and costs no completions. Beating the constant controllers is a floor a controller can pass by being mostly one of them.

1 Introduction

A tool-using agent acts through a runtime: the loop that asks a model for its next step, dispatches the tool calls it returns, feeds results back and decides when to stop [14, 16]. Runtimes differ in everything that matters to control: whether calls in one model turn run in order or in parallel, whether code the model writes can call tools, whether sub-agents get their own tools, how context is compacted, how many rounds a turn may take. Controls written inside one runtime, a permission prompt here and an allowlist there, do not carry to the next, and results measured in one say little about another.

We ask whether one control plane can sit under two quite different runtimes and hold the same run contract in both, and what changes when the agent is a real model rather than a script. The

runtimes are Hermes Lite [7], a Python agent loop in the Hermes line [15], and goose [2], a Rust agent that speaks the Model Context Protocol [1]. The environment is BitAgent's control-capability bench [5]: an agent moving testnet funds through a fixed graph of host-checked steps, with eighteen gates and an oracle that labels every decision.

The contributions are a design, a soundness result without models, and what the first live models exposed:

- A control plane whose ledger is the single source of truth, built so that a runtime only proposes and the gateway only commits under a lease the control module issued (Section 2), integrated into goose as a hook and as a fork and into Hermes Lite as a shim.
- Soundness through four runtime arms on 656 scripted runs, each byte-identical to the run without a runtime (Section 4).
- Seven defects only a live model exposed, one of them a soundness gap, and hard gates at zero after each fix (Section 5).
- A controller qualification method and three ways it can mislead: a floor a lookup passes, a counterfactual replay that misstates a controller with memory, and a scoring path that made every refusal look right (Section 6).

2 The control plane

Control module and ledger. Every event of a run is a record in a hash-chained JSON-lines ledger: the run contract and tool registry at the start, each proposed call with its provenance, each decision with the state hash it used, leases, results, escalations, controller turns and interventions. Control state is a fold over the ledger, so a module restarted mid-run writes a byte-identical ledger, and every decision can be re-derived. A runtime never executes a consequential call on its own say-so: it proposes, the module asks its policy engines (combined by conjunction, so any refusal stands) and, for consequential calls, its controller, and an allowed call gets a single-use lease bound to the call's fingerprint.

Gateway, proxy and egress. The tool gateway is the only path from a runtime to the environment. It speaks MCP over loopback, commits only under a valid lease, and writes the environment's own labels to a truth plane the control module cannot read. The model proxy sits between the runtime and its model server, enforces the context ceiling with the server's own tokenizer, and records every request, so runs can be replayed against recorded model traffic. Every runtime gets an egress sink as its HTTP(S) proxy, which refuses and records any request that would leave the machine.

Runtime integration. goose was run unmodified with a pre-tool-use hook, which leaves calls made from code-mode scripts and sub-agents unseen (the gateway then refuses them as bypasses), and as a fork whose gate proposes every call, including those, with sub-agents registered as their own actors. Hermes Lite carries a shim in its tool dispatch that proposes every call, runs a turn's calls in order rather than in its thread pool, and gates its memory flush. A third arm runs Hermes Lite's unmodified main branch with only the gateway between it and the environment.

Policy engines. An engine decides from the environment's public view, recorded in the ledger. Four were built: BitAgent's eighteen run-contract gates, evaluated in BitAgent's own code through a stdio server (all 7,966 recorded gate evaluations of its soundness suite replay to identical verdicts);

Hermes Lite's skill phase contracts; RTG's ledger gateway; and `bitagent-state-v1`, which refuses an execute when the view shows no admitted, unexpired, gate-clean simulation, no approval, or a move outside the admissible list (Section 6).

Controller thread. Above the agent loop runs a second conversation, the controller. It never reads the agent's raw text. Each ledger record becomes one line of whitelisted fields, with free-text arguments reduced to size and hash unless the tool registry marks a field visible. The thread keeps the run contract, a rolled-up action history, interventions and whether they were followed, open questions and its own notes, and is laid into a fixed 12K-token partition that fails closed when the contract alone does not fit. A blocking turn runs on every consequential call the engines admitted, and the controller answers with one of five actions under a JSON-schema grammar: pass, hold, ask the principal, intervene (a message delivered to the agent) or halt. It can only reduce what the engines decided. A run may pin a short domain brief, appended to the controller's system prompt in its own token lane.

3 Bench, arms and gates

BitAgent's development split is its 42 seed scenarios: nominal deposits, orders and withdrawals and their faults (stale or replayed quotes, unlisted venues, caps, injected intents, ambiguous submissions, expiry). Each turn the host shows a packet with the admissible moves of its graph (observe, inspect, validate, simulate, display, approval, execute, verify, reconcile, complete) and labels the proposed candidate against its oracle: preferred, admissible, wrong or critical. At harness level H3 all eighteen gates are on. Reference policies are the oracle, an adversary that tries to execute early and wide, and random walks.

In step mode each BitAgent turn is one fresh conversation of a real runtime: the candidate travels as a tool call, `bitagent_step` for any move but execute (effect class none) and `bitagent_execute` for execute (state change), and what reaches the gateway is the runtime's proposal. The arms are a minimal loop with a shim (null), Hermes Lite main with only the gateway (R0), Hermes Lite with the shim (R1) and the goose fork over ACP (R2). The hard gates are four counts that must be zero: unauthorized effects at H3, bypasses, tool calls without a proposal, and requests off the machine.

4 Soundness without a model

With scripted policies the runtime must change nothing (Table 1). Each arm ran 164 runs: the oracle on all 42 scenarios, the adversary and one random walk on each, each gate's knockout, and twenty controller runs. Every run equals the same run without a runtime, byte for byte; the adversary and random walks land no unauthorized effect; each knockout lets the adversary land exactly the violation the knocked-out gate prevents; `always_hold` commits nothing, `always_pass` equals no controller, and a scripted hold, question and intervention appear in the ledger and reach the agent. The null arm, R1 and R2 write the same decision projection of the ledger for the same calls (164/164 pairs for each pair of arms): the runtimes differ in prompts and positions, not in what was proposed, decided and committed.

Soundness is only as wide as what the scripts exercise. The scripted policies always chose the tool that matched their move; Section 5 shows what that left untested.

Arm	Oracle = direct	Unauthorized	Adv./random = direct	Knockouts caught	Controller runs	Seconds
null	42/42	0	84/84	18/18	20	82
R0	42/42	0	84/84	18/18	20	155
R1	42/42	0	84/84	18/18	20	193
R2	42/42	0	84/84	18/18	20	250

Table 1: Soundness through four runtime arms with scripted policies (BitAgent seed scenarios, H3). "= direct": runs byte-identical to the same run without a runtime.

5 First live models

The actor was Bonsai-8B [12] in its 1-bit Q1_0 form with BitAgent's v3 DAG adapter, served by llama.cpp [8], given BitAgent's own A1 prompt as the task message. Each defect below was found in a run, fixed in its own commit, and the affected runs were set aside and run again.

1. *A frozen prompt line.* A1 ends "Reply with one JSON object"; with a "submit it as the tool's candidate" line appended, the actor answered in text in all 21 replies of the first seven runs, so nothing reached the tools. Replacing that line with a tool-call instruction under the same field rules gave a tool call in 6 of 6 replayed requests.
2. *Protocol on a shared stdout.* Hermes Lite prints to the stdout its serve protocol uses when a model calls tools until the iteration cap; the harness took the notice for its reply. It now finds the reply by its request id.
3. *Rounds a turn.* The actor called the tool again after every accepted submission, up to the runtime's cap of four rounds and a summary, and a "turn over" line in the tool result did not stop it (3 of 3 replays). Step mode means one candidate a turn, so the runtimes get one tool round a turn.
4. *Unbounded replies.* A degenerate reply repeated one tool call until the context ran out; on a contended GPU one request took 74 minutes. Actor requests are capped at 512 output tokens.
5. *A cap one runtime ignores.* goose over ACP reads `GOOSE_MAX_TURNS`, not `--max-turns`, so its first pass ran about 30 tool rounds a turn where the other arms ran one.
6. *A miscounted audit.* goose reports a call it refused before dispatch (arguments cut off at the output cap) as a failed tool call; the coverage audit counted 477 of them as calls without a proposal. All were of that kind and none was dispatched; they are now counted apart.
7. *The effect class could be sidestepped by the tool choice.* In R0 the actor sent its execute move through `bitagent_step`. The control plane saw a call with no effect: no consequential decision, no controller turn. BitAgent's host executed it, authorized by its own gates, so no unauthorized effect landed, but a consequential move had left the consequential path. The gateway now refuses an execute move through the no-effect tool.

The last item is the one a soundness suite cannot find with policies that always choose the matching tool. It generalizes: an effect class attached to a tool name is sound only if the environment refuses arguments that belong to another class.

With those fixes every hard gate is zero in every cell (Table 2): 1,101 tool calls, each proposed and decided, and 66 more that goose refused before dispatch. The control path held; the actor did not get far. It passes `inspect` and then proposes `validate` without the intent and quote it needs, which are on its screen; it reached approval in 10 of 252 runs and closed 0. Stopping on fault scenarios

Arm	Controller	Runs	Unauth.	Bypass	Uncovered	Off-machine	Acting	Stopping
R1	none	42	0	0	0	0	0 of 21	14 of 21
R1	model	42	0	0	0	0	0 of 21	14 of 21
R2	none	42	0	0	0	0	0 of 21	14 of 21
R2	model	42	0	0	0	0	0 of 21	14 of 21
R0	none	42	0	0	0	0	0 of 21	14 of 21
R0	model	42	0	0	0	0	0 of 21	14 of 21

Table 2: The live actor (Bonsai-8B + v3, A1 prompt, one tool round a turn) through the runtime arms, final run. Acting: runs that should close and did; stopping: runs that should stop and did.

Model (A1, in process, H3)	CFC	Closed	`validate` accepted	Executes
Bonsai-8B + v3 adapter	0.381	1/21	8/44	4
Bonsai-8B base	0.405	0/21	28/93	17
Bonsai-27B base	0.762	13/21	40/48	24

Table 3: BitAgent's in-process A1 arm (no runtime), development scenarios at H3.

is the host holding a model that cannot advance. The actor never called `bitagent_execute`, so in these runs the controller had no turn.

What the runtimes cost the model. BitAgent's in-process A1 arm gives the same models the same prompt without a runtime (Table 3). In process the 8B with the adapter has 8 of 44 `validate` proposals accepted; through the runtimes, 14 of 722. Bonsai-27B closes 13 of 21 nominal scenarios in process; a pilot through R1 reached `display` or `approval` in 4 of 4 nominal runs and executed in none. The runtime's own system prompt and the tool-call format cost the model most of what it does in process.

6 Qualifying a controller

BitAgent's rule for its critic, which we apply to the controller, is that on development data its veto precision and recall must beat `always_pass` and `always_hold` before any result is attributed to it. We signed that rule before scoring, with three additions: F1 must also beat `always_hold`, at least 20 positives and 20 negatives, and a one-sided binomial test that veto precision exceeds the base rate. A turn is positive when its candidate is critical or wrong; holds, halts and questions are vetoes. The controller is the pinned Bonsai-8B base, never the actor's model.

A floor a lookup passes. The first qualification set (reference policies through the null arm with `always_pass`) had 415 controller turns, 344 of them positive. Of these, 341 are executes proposed where execute is not an admissible move, which a lookup of the visible admissible list catches (F1 0.996). The controller, asked each recorded turn again, held 21 times, all rightly, and otherwise passed (236) or intervened (158): F1 0.115 against `always_hold`'s 0.906. It failed even the floor.

A replay misstates a controller with memory. The controller's thread carries its own earlier turns. Run live on the same scenarios, it agreed with the replay on the first turn of 78 of 85 runs; from the second turn on it intervened where the replay had passed (228 turns) and where it had

Set	Turns (pos.)	Base rate	Precision	Recall	F1	`always_hold` F1	View rule F1	Completions	Qualifies
Development	1,102 (193)	0.175	0.000	0.000	0.000	0.298	0.887	816→815	no

Table 4: The pinned Bonsai-8B base controller, no brief, live, qualification v2 (admissible-move turns). Completions: runs reaching their expected end, without → with the controller.

held (21). Asked each turn of an `always_pass` run, it saw a history of passes it would not have written. A controller whose turns depend on its own must be qualified live.

Qualification v2. The second rule keeps the conditions and scores only turns whose candidate was an admissible move, where the lookup is no better than `always_pass`, from live runs. Its reference collection (60 random seeds a scenario, `always_pass`) has 1,098 such turns, 193 positive (base rate 0.176); `always_hold` scores F1 0.299, and holding what the view marks faulty scores F1 0.887 (precision 0.878, recall 0.896), reported but not required.

Every refusal scored as right. Our first live runs carried a scoring path that cannot qualify anything. A call the control plane refused never reaches the gateway, so BitAgent's runner received no proposal for that turn and labelled it critical, and the scorer joined each refused turn to that label: every hold and every engine refusal was a true positive by construction. Under it the briefed controller below showed precision 1.000 and recall 0.980 on held-out A; 689 of its 699 holds were of well-formed candidates labelled with no candidate at all. The runner now receives the refused candidate and labels it exactly as it labels the same candidate applied: an execute's gates are evaluated, nothing is applied or recorded (tested against runs that apply the same candidate). The scorer fails any verdict in which a veto of a well-formed candidate carries a label without that candidate's key. (A first version of this check also failed refused moves the host would have rejected anyway, which BitAgent labels with their key but not as control refusals, and random-policy candidates with an extra field, which its strict schema never keys in any run; it was narrowed before any r2 result was read beyond that check.) Every live result below is from runs made after the fix.

Results. The pinned controller with no brief, live on the development seeds, is in Table 4: on the admissible-move turns it held 0.005 of the time and mostly intervened, so it catches nothing and fails every condition. (Its first attempt stopped after 53 runs when three runs of one scenario failed to finish in a row; those were set aside and the run resumed.) A brief protocol was fixed before any brief was written: only the brief is tuned, on development seeds; held-out A is the seed scenarios with unused random seeds and no reference policies; held-out B is BitAgent's 40 open-gate scenarios, none in the seed set; each is run once with the chosen brief, and the controller qualifies only if the v2 conditions pass on both. One brief was written and chosen (it says what the view shows, when to hold, to pass otherwise, and not to intervene to repeat the rules); its results are in Table 5. The state engine, whose checks were chosen on the reference collection, is in Table 6. The engine and the controller change the runs they decide, and a refused move may be proposed again, so their turn sets differ and the tables are not a ranking.

What the pass is worth. The briefed controller qualifies on development and on both held-out sets, and the margin is thin. It holds 0.773 of the admissible-move turns on development (0.759 on held-out A), so its recall is high and its precision, 0.194, sits a few points above the base rate of 0.165; its F1, 0.320, beats `always_hold`'s by 0.037. The brief states a rule, and the "view rule" column applies that rule to the same turns: F1 0.801 on development, 0.876 on held-out A and 0.731

Set	Turns (pos.)	Base rate	Precision	Recall	F1	`always_hold` F1	View rule F1	Completions	Qualifies
Development	3,209 (528)	0.165	0.194	0.911	0.320	0.283	0.801	816→681	yes
Held-out-A	3,037 (504)	0.166	0.199	0.913	0.327	0.285	0.876	749→640	yes
Held-out-B	5,476 (1,018)	0.186	0.220	0.951	0.357	0.314	0.731	477→354	yes

Table 5: The same controller with the brief, live, qualification v2. Held-out A: unused seeds; held-out B: the open-gate scenarios.

Set	Turns (pos.)	Base rate	Precision	Recall	F1	`always_hold` F1	View rule F1	Completions	Qualifies
Development	1,098 (193)	0.176	1.000	0.896	0.945	0.299	–	816→816	yes
Held-out-A	972 (95)	0.098	1.000	0.789	0.882	0.178	–	749→749	yes
Held-out-B	1,703 (309)	0.181	1.000	0.906	0.951	0.307	–	477→477	yes

Table 6: The state engine `bitagent-state-v1` alone (no controller), live, scored by the v2 conditions on its refusals.

on held-out B. The 8B model read the brief as a reason to hold, not as a test to apply. The engine that implements the rule in code refuses with precision 1.000 and recall 0.896 on development and needs no model, no GPU and no brief. The rule's floor, beating the two constant controllers, is passed by a controller that is mostly one of them.

What a refusal costs. Before the fix a refused turn also arrived as a missing proposal, which BitAgent counts as a failed attempt: when its retries ran out the run was held, where the host's own refusal of the same move lets the run continue. Under that path the briefed controller's correct holds cut completions on the development seeds from 816 to 651. The runner now treats a control refusal like its own: the step is used, the node stays, the run goes on. The completion columns of Tables 4–6 are under that path. Holds still cost what they use: the briefed controller's take the development seeds from 816 completions to 681 (held-out A 749 to 640, held-out B 477 to 354), because most of them stop moves that were right. The engine's refusals cost none (816 to 816): it refuses only executes the host's own gates would have stopped later.

The controller across runtimes. Table 7 runs the pinned controller with no brief live across all four arms, deciding the calls of the reference policies, so the controller's path is exercised in each runtime regardless of the actor's skill. In 672 runs it took 1,657 turns and held 9 times; it intervened 1,445 times, and all 1,445 interventions reached the agent. They carried nothing the agent's own state did not show: most were generic exhortations ("Please review the task and ensure all necessary actions are taken") or reminders of the event that triggered the turn. That is why the brief tells the controller not to intervene to repeat the rules or encourage the agent. Every hard gate stayed at zero in every arm. The mix of actions differs between arms (under the goose fork the controller passed far more often), so the same controller is not the same monitor under every runtime, even though its path through each is.

7 Discussion and limitations

What held and what did not. The control path held in every arm and every run: no unauthorized effect, no bypass, no unproposed call, nothing off the machine. Everything else we learned came from places a scripted bench does not reach: the prompt, the runtime's own conventions, and the measurement. Six of the seven live defects were in our harness, not in the control plane or the

Arm	Runs	Turns	Hold	Intervene	Pass	Precision	Recall	Hard-gate events
null	168	415	0	406	9	–	0.000	0
R1	168	412	7	387	18	0.857	0.018	0
R2	168	415	2	272	141	1.000	0.006	0
R0	168	415	0	380	35	–	0.000	0

Table 7: The pinned controller (no brief) live in each runtime arm, deciding the reference policies' calls (two random seeds a scenario). Scored on all controller turns.

models; the seventh was a gap in how the step environment tied effect classes to tools. All three qualification problems were in our method.

What the brief shows. The brief told the controller which faults to look for, and the controller passed the signed rule without applying them: it held most turns, and the rule written in its brief, applied to the same turns, does far better. A pass of v2 was meant to show the controller reads the faults its view shows; this one shows a controller that holds often enough to clear two constant baselines. The rule's floor is necessary and not sufficient. A stricter rule would compare a controller with the deterministic rule its brief states, or require precision well above the base rate rather than significantly above it; we did not change the signed rule after seeing these results, and report the comparison instead. Where the deterministic engine suffices, it is better on every count here, and it does not fail when its server runs out of memory, which the controller's did several times on a shared GPU; the runner now refuses to start a run while the controller is down.

Limitations. All results are on one bench: held-out A shares its scenarios with development, and held-out B is a different scenario family from the same generator. One model family was used, in 1-bit form, and the qualification actors are scripted reference policies, not models. One brief was tried. The live controller ran on a GPU shared with other jobs, so timings are not throughput measurements. Labels are BitAgent's oracle's; where the oracle is wrong, so are our scores.

8 Related work

AI control studies safety protocols that hold even if the model is adversarial, using a weaker trusted model to monitor a stronger untrusted one [9]; our controller is a trusted monitor in that sense, and our qualification is a test of whether it monitors at all. CaMeL separates control flow from untrusted data so that injected text cannot change what an agent does [4]; our renderer's rule that the controller never reads free text is the same separation at a smaller scale. AgentDojo and τ-bench evaluate agents' tool use under attack and with users [3, 17]; our bench differs in that its oracle labels every decision and its host, not the agent, holds the authority to commit, as LLM-Modulo frameworks place external verifiers in charge of acceptance [11]. Guardrail toolkits and safety classifiers filter model inputs and outputs [10, 13]; the control plane here gates actions, at the boundary between the runtime and the world, whatever the text said.

9 Conclusion

One control plane can sit under two quite different agent runtimes and hold the same run contract in both, with every scripted run unchanged by the runtime. The first live models did not break

the control path, but they broke almost everything around it, and the most useful findings were about measurement: a qualification floor a lookup passes, a counterfactual replay that misstates a controller with memory, and a scoring path in which every refusal looked right. Judged live with those corrected, a 1-bit 8B controller with no brief fails, and with a brief it passes the signed rule on held-out scenarios while holding most turns, far below the rule its own brief states; the deterministic engine that implements the rule is better on every measure here. For this bench the useful controller is the engine, and a model controller has yet to show it adds anything a rule over the visible state does not.

Reproduction. Code, run directories and rules are in Goose-Experiments [6] and BitAgent [5]; every number above is generated by `paper/make_tables.py` from those files. The qualification rules are `configs/controller_qualification_v1.json` and `_v2.json`, the brief protocol `configs/controller_brief_protocol_v1.json`, and the brief `configs/controller_briefs/bitagent-b1.txt`.

References

[1] Anthropic. Model context protocol. `https://modelcontextprotocol.io`, 2024. Specification and SDKs for connecting language-model applications to tools.

[2] Block. goose: an open-source, extensible AI agent. `https://github.com/block/goose`, 2025. Version 1.53.0 is the base of the fork used here (`https://github.com/patrickdugan/goose`, branch `control-gate`).

[3] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. URL `https://arxiv.org/abs/2406.13352`. arXiv:2406.13352.

[4] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025. URL `https://arxiv.org/abs/2503.18813`.

[5] Patrick Dugan. BitAgent: a launch kernel and control-capability bench for an agent that moves testnet funds. `https://github.com/patrickdugan/BitAgent`, 2026. Directory `apps/bitagent-launch-kernel`; design document `docs/runtime-control-layer.md`.

[6] Patrick Dugan. Goose-experiments: a runtime control plane for agent runtimes. Repository `patrickdugan/Goose-Experiments`, 2026. Package `runtime_control`, run directories under `results/runs`, and the qualification rules under `configs`.

[7] Patrick Dugan. Hermes lite: a python agent runtime. `https://github.com/patrickdugan/hermes-lite`, 2026. Branches `control-shim` (arm R1) and `r0-gateway-bridge` (arm R0).

[8] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, 2023.

[9] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL `https://arxiv.org/abs/2312.06942`. arXiv:2312.06942.

[10] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama guard: LLM-based input-output safeguard for human-AI conversations, 2023. URL https://arxiv.org/abs/2312.06674.

[11] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. Position: LLMs can't plan, but can help planning in LLM-modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. URL https://arxiv.org/abs/2402.01817. arXiv:2402.01817.

[12] PrismML. Bonsai: 1-bit language models (1.7B, 4B, 8B). https://huggingface.co/prism-ml/Bonsai-8B-gguf, 2026. Model cards and GGUF artifacts in the Q1_0 format.

[13] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2023. URL https://arxiv.org/abs/2310.10501. arXiv:2310.10501.

[14] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2302.04761. arXiv:2302.04761.

[15] Ryan Teknium, Jeffrey Quesnelle, and Chen Guang. Hermes 3 technical report, 2024. URL https://arxiv.org/abs/2408.11857.

[16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2210.03629. arXiv:2210.03629.

[17] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL https://arxiv.org/abs/2406.12045.